

Language



Автор сайта



Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



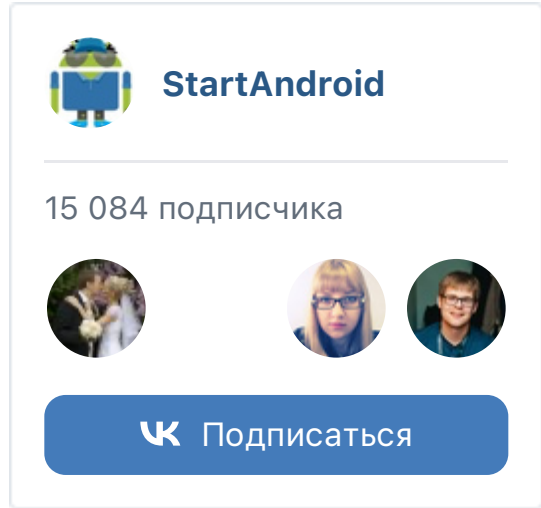
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

[Яндекс](#)
410011180491924

[WebMoney](#)
R248743991365
Z551306702056

[Paypal](#)

Урок 16. Hilt. Основы

27 августа 2021

В этом уроке подключаем Hilt к проекту и используем его для инджекта объектов

На прошлом уроке мы на простом примере рассмотрели, как Hilt работает изнутри. Теперь давайте разберемся, как использовать Hilt в приложении.

В новом проекте сделаем пример того же самого приложения с двумя Activity (OrderActivity и UserActivity) и фрагментом (OrderFragment). Но теперь Hilt все свои классы будет создавать сам.

Подключение

В build.gradle файла проекта в секции dependencies добавляем Hilt зависимость:

```
1 | classpath 'com.google.dagger:hilt-android-gradle-plugin:2.37'
```

В build.gradle файла модуля подключаем плагины kapt и Hilt в секции plugins:

```
1 | id 'kotlin-kapt'
2 | id 'dagger.hilt.android.plugin'
```

а также Hilt зависимости в секции dependencies:

```
1 | implementation 'com.google.dagger:hilt-android:2.37'
2 | kapt 'com.google.dagger:hilt-compiler:2.37'
```

Application

Интеграцию Hilt в приложение начнем с App класса. Для этого просто помечаем наш App класс аннотацией HiltAndroidApp:

```
1 | @HiltAndroidApp
2 | class App: Application() {
3 |
4 | }
```

Это все, что от нас требуется. А Hilt сделает все то, что мы делали на прошлом уроке: создаст AppComponent и специальный Hilt_App класс, который станет супер классом для нашего App. При этом нам самим ничего наследовать не придется. Все будет сделано автоматически.

Мы просто добавляем в App переменные, которые нам нужны, и помечаем их аннотацией Inject.

```
1 | @HiltAndroidApp
2 | class App: Application() {
3 |
4 |     @Inject lateinit var databaseHelper: DatabaseHelper
5 |
6 |     override fun onCreate() {
7 |         super.onCreate()
8 |         Log.d(TAG, "databaseHelper = $databaseHelper")
9 |     }
10 |
11 | }
```

Инджект происходит под капотом.

Activity

Чтобы Hilt взял в работу наше Activity, его необходимо пометить аннотацией @AndroidEntryPoint:

```
1 | @AndroidEntryPoint
2 | class OrderActivity : AppCompatActivity() {
3 |
4 |     @Inject
5 |     lateinit var repository: OrderRepository
6 |
7 |     override fun onCreate(savedInstanceState: Bundle?) {
8 |         super.onCreate(savedInstanceState)
9 |         setContentView(R.layout.activity_order)
10 |
11 |         Log.d(TAG, "repository = $repository")
12 |     }
13 | }
```

Hilt добавит в ActivityComponent инджект метод для этого Activity. А также создаст super класс для этого Activity, куда поместит весь даггер код. Нам самим явно ничего наследовать не надо. Мы только используем аннотацию Inject, чтобы попросить объекты у даггера.

AndroidEntryPoint

AndroidEntryPoint - универсальная аннотация, которой мы можем помечать Android сущности: Activity, фрагмент, сервис, View и Broadcast Receiver. Для Hilt - это сигнал, что он должен применить свою даггер-магию к этому объекту.

Добавим AndroidEntryPoint к UserActivity:

```
1 | @AndroidEntryPoint
2 | class UserActivity: AppCompatActivity() {
3 |
4 |     @Inject
5 |     lateinit var repository: UserRepository
6 |
7 |     override fun onCreate(savedInstanceState: Bundle?) {
8 |         super.onCreate(savedInstanceState)
9 |         setContentView(R.layout.activity_user)
10 |         Log.d(TAG, "repository = $repository")
11 |     }
12 |
13 | }
```

и к OrderFragment:

```
1 | @AndroidEntryPoint
2 | class OrderFragment : Fragment() {
3 |
4 |     @Inject
5 |     lateinit var repository: OrderRepository
6 |
7 |     override fun onCreate(savedInstanceState: Bundle?) {
8 |         super.onCreate(savedInstanceState)
9 |         Log.d(TAG, "fragment repository = $repository")
10 |     }
11 |
12 |     override fun onCreateView(inflater: LayoutInflater,
13 |                               container: ViewGroup?,
14 |                               savedInstanceState: Bundle?): View? {
15 |         return inflater.inflate(R.layout.fragment_order, container, false)
16 |     }
17 |
18 | }
```

Под капотом Hilt делает все необходимое, и мы получаем возможность инджектить нужные нам объекты без лишнего кода.

Назад

Вперёд

Комментарии

- # **BroadcastReceiver** — dev_course 01.12.2021 17:18
Стоит упомянуть отдельный нюанс с BroadcastReceiver
Unlike the other supported Android classes, BroadcastReceivers do not have their own Dagger component and are instead simply injected from the SingletonComponent. ↩
https://dagger.dev/hilt/android-entry-point#fnref:1

Ответить
- # **VS code** — Vadim 09.02.2022 15:21
аналогично есть нюанс с content provider

Ответить
- # **RE: BroadcastReceiver** — Евгений 03.05.2023 10:16
Список поддерживаемых аннотаций @AndroidEntryPoint компонентов по ссылке:
https://developer.android.com/training/dependency-injection/hilt-android#android-classes

Ответить

🔄 Обновить список комментариев
📄 RSS лента комментариев этой записи

Добавить комментарий

Тема

😄 😊 😌 😍 😎 😏 😐 😑 😒 😓 😔 😕 😖 😗 😘 😙 😚 😛 😜 😝 😞 😟 😠 😡 😢 😣 😤 😥 😦 😧 😨 😩 😪 😫 😬 😭 😮 😯 😰 😱 😲 😳 😴 😵 😶 😷 😸 😹 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿

Отправить

